

Micrograd & MLP — Complete Notes

Working through Andrej Karpathy's *Neural Networks: Zero to Hero* Notebooks:
github.com/sauravtiitj/neural-networks-zero-to-hero

Part 1 — Micrograd: The Value Class

What is Micrograd?

A tiny Autograd engine. Implements backpropagation (reverse-mode autodiff) over a dynamically built DAG and a small neural networks library on top of it with a PyTorch-like API. Both are tiny, with about 100 and 50 lines of code respectively. The DAG only operates over scalar values — we chop up each neuron into all of its individual tiny adds and multiplies. Enough to build entire deep neural nets doing binary classification.

What Value Does That a Float Cannot

A plain Python float stores one thing: a number.

```
x = 3.0    # just a number. no history. no gradient.
```

A `Value` stores three things:

```
a = Value(-4.0)
# a.data = -4.0    → the actual number
# a.grad = 0       → gradient (how much output changes if we nudge a)
# a._prev = {}      → which Values created this one (computation history)
```

That third thing — `_prev` — is what makes learning possible.

Basic Usage

```
from micrograd.engine import Value # stores scalar value, gradient, computation
history

a = Value(-4.0)                    # input node a = -4.0
print(a)                          # Value(data=-4.0, grad=0)
print(a.data)                     # -4.0
print(a.grad)                    # 0

b = Value(2.0)                    # input node b = 2.0

c = a + b                          # c = -2, graph records: c came from a + b
d = a * b + b**3                  # d = (a×b) + b³

c += c + 1                        # expands computation graph
c += 1 + c + (-a)

d += d * 2 + (b + a).relu()       # relu: if x>0 output x else 0. b+a=-2 →
relu=0
d += 3 * d + (b - a).relu()
```

```

e = c - d
f = e**2
g = f / 2.0
g += 10.0 / f

print(f'{g.data:.4f}')          # 24.7041 — forward pass result

g.backward()                    # run backpropagation

print(f'{a.grad:.4f}')          # 138.8338 — dg/da
print(f'{b.grad:.4f}')          # 645.5773 — dg/db

```

The Computation Graph

Every operation builds a graph — called a DAG (Directed Acyclic Graph):

```

      → c →
a →    → e → f → g
      → d →

```

Every arrow is an operation. Every node is a Value. The graph records the entire story of how the final output was computed.

Chain Rule:

```

dg/da = dg/df × df/de × (de/dc × dc/da + de/dd × dd/da)

```

Derivatives — Back to Basics

```

def f(x):
    return 3*x**2 - 4*x + 5

```

The derivative tells us: if we nudge x slightly, how much does $f(x)$ change?

```

h = 0.0001    # tiny nudge
x = 3.0

# numerical derivative
slope = (f(x + h) - f(x)) / h
# as h → 0, this approaches the true derivative

```

This is exactly what autograd does — but analytically, not numerically.

Why Gradients Use += Not =

Suppose a contributes to the output through two different paths:

```

      b ---- \
      /       \
a ---- \       / ----> o
      \       /
      c ---- /

```

Both paths contribute to do/da:

$$\frac{do}{da} = \frac{do}{db} \frac{db}{da} + \frac{do}{dc} \frac{dc}{da}$$

Notice the PLUS sign. Both contributions must be added — not replaced.

Therefore:

```
a.grad += contribution1
a.grad += contribution2
# ADD contributions instead of replacing them
```

If you used = instead of += you would lose one path's gradient entirely.

```
self.grad += something
# means: self.grad = self.grad + something
```

Topological Sorting — The Correct Order

Manual backprop only works if we call `_backward()` in the right order:

```
# CORRECT ORDER:
o._backward()
n._backward()
x1w1x2w2._backward()
x1w1._backward()
x2w2._backward()
```

Problem: `x1w1._backward()` needs `x1w1.grad` to already exist. Therefore `x1w1x2w2._backward()` must run first.

Topological order (forward):

```
x1 → w1 → x1w1 → x2 → w2 → x2w2 → x1w1x2w2 → b → n → o
```

Backprop uses `reversed(topo)`:

```
o → n → b → x1w1x2w2 → x2w2 → w2 → x2 → x1w1 → w1 → x1
```

Parents appear before children in forward order. Gradients flow correctly in reverse order.

The Complete Mental Model

Forward Pass:

```
Create nodes
Store parents (_prev)
Store operation (+, *, tanh)
Store backward rule (_backward)
Compute values
```

Backward Pass:

```
Set output gradient = 1
Build topological order
Reverse topological order
```

```
Call _backward() on every node
Each node sends gradient contributions to its parents
Gradients accumulate using +=
```

One sentence summary:

Each node knows how to send gradients to its parents (`_backward`). Topological sorting simply finds the correct order to call those functions.

How Value Connects to Neuron

When a Neuron computes:

```
act = sum((wi*xi for wi, xi in zip(self.w, x)), self.b)
out = act.tanh()
```

Every `wi`, `xi`, `self.b`, `act`, `out` — all are `Value` objects. Every `*`, `+`, `.tanh()` adds a node to the computation graph.

When `loss.backward()` is called — gradients flow back through every node, all the way to `wi` and `self.b`.

That is how the network learns. Not magic. Just Value objects remembering their history.

Part 2 — The MLP: Complete Workflow

The Four Classes

Class	What it is	Analogy
Value	A number with memory	The foundation
Neuron	One calculation unit	One interviewer scoring a candidate
Layer	A row of neurons	A panel of interviewers
MLP	Multiple layers stacked	Multiple interview rounds

Call chain:

```
MLP → calls → Layer → calls → Neuron → uses → Value
```

Neuron

```
class Neuron:
    def __init__(self, nin):
        # nin = number of inputs
        # one weight per input – randomly initialised between -1 and 1
        # these are what the network LEARNS during training
        self.w = [Value(random.uniform(-1,1)) for _ in range(nin)]
```

```

# one bias – the neuron's default value before seeing any input
self.b = Value(random.uniform(-1,1))

def __call__(self, x):
    # w * x + b
    # zip pairs each weight with its input
    # sum adds them all up starting from self.b
    # formula: w[0]*x[0] + w[1]*x[1] + ... + bias
    act = sum((wi*xi for wi, xi in zip(self.w, x)), self.b)
    # tanh squashes output to between -1 and +1
    # prevents numbers exploding as layers stack
    # introduces non-linearity – critical for learning complex patterns
    out = act.tanh()
    return out

def parameters(self):
    # all learnable values = weights + bias
    # training loop uses this to know what to update
    return self.w + [self.b]

```

Layer

class Layer:

```

def __init__(self, nin, nout):
    # nin = inputs coming in
    # nout = number of neurons in this layer
    # all neurons receive the SAME input
    # but each has own weights → each learns something different
    self.neurons = [Neuron(nin) for _ in range(nout)]

def __call__(self, x):
    # run every neuron on the same input x
    # each produces one output value
    outs = [n(x) for n in self.neurons]
    # if 1 neuron: return value directly, not wrapped in list
    # if multiple: return full list
    return outs[0] if len(outs) == 1 else outs

def parameters(self):
    # collect parameters from ALL neurons into one flat list
    # nested loop in one line: for each neuron → get its parameters
    return [p for neuron in self.neurons for p in neuron.parameters()]

```

Layer is the intermediary between MLP and Neuron:

MLP calls layer(x)

- Layer loops through neurons
 - Each neuron returns one Value
- Layer collects into list
- Returns list to MLP

MLP

class MLP:

```

def __init__(self, nin, nouts):
    # nin = inputs to the whole network

```

```

# nouts = list of layer sizes e.g. [4, 4, 1]
#
# MLP(3, [4, 4, 1]):
#   3 inputs → Layer of 4 → Layer of 4 → Layer of 1 (output)

# full size list: [3, 4, 4, 1]
sz = [nin] + nouts
# pair consecutive sizes to define each layer
# Layer 0: 3→4, Layer 1: 4→4, Layer 2: 4→1
self.layers = [Layer(sz[i], sz[i+1]) for i in range(len(nouts))]

def __call__(self, x):
    # forward pass – input flows through every layer in sequence
    # output of each layer becomes input to next
    for layer in self.layers:
        x = layer(x)
    return x

def parameters(self):
    # collect ALL parameters from ALL layers into one flat list
    # training loop iterates over this to update weights
    return [p for layer in self.layers for p in layer.parameters()]

```

Tracing a Real Input: MLP(3, [4, 4, 1])

Input: $x = [2.0, 3.0, -1.0]$

```

n([2.0, 3.0, -1.0])
  ↓
MLP.__call__(x)
  |   for layer in self.layers:
  |   ↓
Layer 1.__call__([2.0, 3.0, -1.0])
  |   for n in self.neurons:
  |   |   → Neuron1([2.0, 3.0, -1.0]) → 0.71
  |   |   → Neuron2([2.0, 3.0, -1.0]) → -0.42
  |   |   → Neuron3([2.0, 3.0, -1.0]) → 0.13
  |   |   → Neuron4([2.0, 3.0, -1.0]) → 0.95
  |   |   └─ returns [0.71, -0.42, 0.13, 0.95]
  |   ↓
Layer 2.__call__([0.71, -0.42, 0.13, 0.95])
  |   |   → Neuron1 → 0.33
  |   |   → Neuron2 → 0.87
  |   |   → Neuron3 → -0.21
  |   |   → Neuron4 → 0.64
  |   |   └─ returns [0.33, 0.87, -0.21, 0.64]
  |   ↓
Layer 3.__call__([0.33, 0.87, -0.21, 0.64])
  |   |   → Neuron1 → 0.54
  |   |   └─ returns 0.54
  |   ↓
MLP returns: 0.54

```

Why Each Class Exists

Neuron — the maths lives here. Every multiplication, addition, tanh happens inside Neuron. Everything else just moves data around.

Layer — runs multiple neurons in parallel. Organises them, feeds same input to all, gathers outputs.

MLP — multiple layers in sequence. Passes x through each layer, letting the representation transform at each step.

The Key Insight

A neural network is just nested loops:

```
for layer in layers:           # MLP loops through layers
    for neuron in layer:       # Layer loops through neurons
        for weight in neuron: # Neuron loops through weights
            output += weight * input
```

Karpathy wraps this in classes so it looks elegant. Underneath — loops inside loops inside loops.

The magic is not the loops. The magic is that the weights learn. Everything else is just loops.

The Training Loop (What Comes Next)

Network predicted 0.54. Correct answer is 1.0. It was wrong.

```
1. Forward pass    → prediction (0.54)
2. Compute loss    →  $(0.54 - 1.0)^2 = 0.21$ 
3. loss.backward() → gradients for every weight in every neuron
4. Update weights  → p.data -= lr * p.grad
5. Zero gradients  → p.grad = 0
6. Repeat          → prediction improves each time
```

The gradient for every weight — computed automatically by Value's autograd engine. Every `wi` and `self.b` gets its gradient. `p.data -= lr * p.grad` nudges each weight in the direction that reduces the loss.

That is the complete picture.

Summary

```
Value    → stores number + gradient + history
Neuron   →  $w \cdot x + b \rightarrow \tanh \rightarrow$  one output Value
Layer    → runs N neurons on same input → N output Values
MLP      → chains layers → transforms input to final prediction
backward → gradients flow back through Value graph to every weight
training → repeat forward + backward + update until loss → 0
```

Notebooks: github.com/sauraviitj/neural-networks-zero-to-hero Playlist: [Andrej Karpathy — Neural Networks: Zero to Hero](#)

